

# Kimia

Fixed-Rate Yield Markets for Solana  
A Composable Yield-Tokenization Stack  
*LitePaper v1.0*

Kimia Labs

support.kimia@gmail.com

2026

## Abstract

Existing Solana yield products are fragmented. Users can earn yield across perps, vaults, lending, and stablecoins, but they cannot easily lock that yield, price it, trade it, or exit before maturity. We present *Kimia*, a Solana-native fixed-rate yield protocol that turns future yield into tradable markets. Users deposit USDC into a delta-neutral vault that targets funding-rate yield, split vault shares into Principal Tokens (PT) and Yield Tokens (YT), trade principal-token exposure through a yield AMM, and lock fixed-rate returns instead of passively waiting for variable yield. Kimia is built as a composable protocol stack: an on-chain perpetuals engine, delta-neutral vault, PT/YT split engine, yield AMM, kUSD settlement layer, and intent router. Together, these programs let users lock, trade, settle, and exit Solana yield positions through one protocol. Kimia starts with funding-rate yield and expands toward a broader vision: making every Solana yield stream tradable.

**Keywords:** perpetual futures · funding rate · delta-neutral · yield tokenization · principal tokens · yield-space AMM · rate-target intent · Solana · Anchor

**Legal Disclaimer.** This White Paper is a proposed technical paper and architecture plan for Kimia v1. It is shared for feedback and discussion, and should not be read as the final protocol architecture, a product guarantee, financial advice, or an offer to buy or sell any token or financial instrument. Kimia involves DeFi, perpetuals, vault accounting, and smart contracts, all of which require extensive testing, audits, risk review, and possible design changes before any production release. Parameters, mechanisms, integrations, and roadmap items described here may change materially as the protocol is developed.

# Contents

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                               | <b>4</b>  |
| <b>2</b> | <b>Theoretical Foundation</b>                     | <b>5</b>  |
| 2.1      | Kimia Funding Kernel . . . . .                    | 5         |
| 2.2      | Delta-Neutral Hedging . . . . .                   | 5         |
| 2.3      | The Reserved-Margin Health Model . . . . .        | 5         |
| 2.4      | The Split-Reward High-Water Guard . . . . .       | 6         |
| 2.5      | Implementation-Side AMM Invariant Guard . . . . . | 6         |
| 2.6      | The Pre-Pause Solvency Predicate . . . . .        | 6         |
| <b>3</b> | <b>Protocol Architecture</b>                      | <b>6</b>  |
| 3.1      | Layered View . . . . .                            | 7         |
| 3.2      | CPI Coupling . . . . .                            | 7         |
| <b>4</b> | <b>Perpetual Engine: kimia-perp</b>               | <b>7</b>  |
| 4.1      | Position Accounting . . . . .                     | 8         |
| 4.2      | Reserved-Margin Convention . . . . .              | 8         |
| 4.3      | Mark Price Clamp . . . . .                        | 9         |
| 4.4      | Liquidation . . . . .                             | 9         |
| 4.5      | Orderbook . . . . .                               | 9         |
| <b>5</b> | <b>Delta Vault</b>                                | <b>10</b> |
| 5.1      | Deposit Mechanics . . . . .                       | 10        |
| 5.2      | NAV and Share Price . . . . .                     | 10        |
| 5.3      | Rebalancing and Pause . . . . .                   | 11        |
| <b>6</b> | <b>Split Engine</b>                               | <b>11</b> |
| 6.1      | Mint and Redeem . . . . .                         | 11        |
| 6.2      | Early Exit . . . . .                              | 11        |
| <b>7</b> | <b>Yield-Space AMM</b>                            | <b>12</b> |
| 7.1      | Pool Parameters . . . . .                         | 12        |
| 7.2      | Time Decay and Convergence . . . . .              | 12        |
| 7.3      | Implementation Notes . . . . .                    | 13        |
| <b>8</b> | <b>Intent Router</b>                              | <b>13</b> |
| 8.1      | Session State Machine . . . . .                   | 13        |
| 8.2      | Why a State Machine? . . . . .                    | 13        |
| <b>9</b> | <b>kUSD Stablecoin Layer</b>                      | <b>13</b> |

|           |                                                               |           |
|-----------|---------------------------------------------------------------|-----------|
| 9.1       | Mint and Redeem . . . . .                                     | 14        |
| 9.2       | Cross-Stable Swap Intents . . . . .                           | 14        |
| 9.3       | Staking Yield . . . . .                                       | 14        |
| <b>10</b> | <b>Risk Engine and Insurance</b>                              | <b>14</b> |
| 10.1      | Health Snapshot . . . . .                                     | 14        |
| 10.2      | Insurance and Crank Incentives . . . . .                      | 14        |
| <b>11</b> | <b>Implementation Notes</b>                                   | <b>15</b> |
| 11.1      | Account Loading and <code>remaining_accounts</code> . . . . . | 15        |
| 11.2      | Zero-Copy Orderbook . . . . .                                 | 15        |
| 11.3      | PDA Authority and CPI Signing . . . . .                       | 15        |
| 11.4      | Lazy Funding and Settlement Order . . . . .                   | 15        |
| 11.5      | Rounding Discipline . . . . .                                 | 16        |
| 11.6      | Permissionless Cranking . . . . .                             | 16        |
| <b>12</b> | <b>Numerical Precisions</b>                                   | <b>16</b> |
| 12.1      | Fixed-Point Math Library . . . . .                            | 17        |
| <b>13</b> | <b>End-to-End Yield-Tokenization Flow</b>                     | <b>17</b> |
| <b>14</b> | <b>Discussion</b>                                             | <b>17</b> |
| 14.1      | Why Solana . . . . .                                          | 17        |
| 14.2      | Composition Surface . . . . .                                 | 18        |
| 14.3      | Security Properties . . . . .                                 | 18        |
| 14.4      | Limitations . . . . .                                         | 18        |
| 14.5      | Position in the Landscape . . . . .                           | 19        |
| <b>15</b> | <b>Conclusion</b>                                             | <b>19</b> |

# 1 Introduction

Solana yield is increasingly deep, but still fragmented. A user can earn yield across perpetual funding, vault strategies, lending markets, liquid staking, and stablecoin reserves; the hard part is turning that variable stream into an asset that can be priced, traded, locked, and exited before maturity. Perpetual funding is the first yield source Kimia targets because it is observable on chain, settles continuously, and already supports a familiar cash-and-carry construction [14, 1]. The funding rate  $f_t$  is the periodic transfer that keeps the perpetual mark price tethered to the underlying oracle: when the perp trades above oracle, longs pay shorts; when it trades below, shorts pay longs. When funding is positive, a hedged spot-long/perp-short position can receive funding while reducing first-order price exposure.

Existing on-chain products usually expose only one part of this stack. Perp venues provide the matching layer [1]; tokenization protocols separate principal from yield [2, 4, 5]; fixed-rate AMMs price claims against maturity [3, 4]; stablecoin systems handle settlement and collateral. A user who wants stablecoin-denominated, term-structured exposure to funding-rate yield must currently:

1. open a spot leg on an AMM,
2. open a matched perp short on a separate venue,
3. monitor the delta and rebalance manually,
4. exit before maturity to crystallize a rate.

Each step leaks slippage, settlement risk, and price exposure. There is no single Solana-native flow that takes a user from deposit to tokenized position to fixed-rate execution with an explicit rate guard.

**Contribution.** We present *Kimia*, a vertically integrated fixed-rate yield stack built as six independently auditable Anchor programs on Solana. Kimia gives the user a tractable primitive: deposit USDC, choose a tenor and a target rate, and let the protocol either execute at the prevailing AMM-implied rate or revert. The realized value of the final position is determined by the vault’s actual NAV trajectory over the tenor and is therefore *not* pre-guaranteed; what is locked at session time is the discount implied by selling PT exposure against the underlying. Internally, the protocol composes:

1. a perp matching engine with on-chain orderbook, lazy funding, and oracle-clamped mark pricing;
2. a delta-neutral vault that pairs an oracle-priced spot leg with a CPI-opened perp short;
3. a tokenization engine that splits vault shares into Principal Tokens (PT) and Yield Tokens (YT);
4. a yield-space AMM that prices PT against the underlying and decays PT into par at maturity;
5. an intent router that verifies multi-transaction yield-tokenization sessions against a user-supplied rate target;
6. a multi-reserve stablecoin (kUSD) with a staking yield channel for settlement and reward distribution.

The combination is a stack with four design properties: *delta-neutral* funding capture, *tokenized* yield separation via PT/YT, an explicit *term-structure* curve through the yield-space AMM, and *verified* multi-step intent execution. Each layer is independently usable; the whole composes into one user-facing primitive.

## 2 Theoretical Foundation

### 2.1 Kimia Funding Kernel

The funding rate  $f_t$  of a perpetual contract is the periodic payment from one side to the other that keeps the perp mark price tethered to its oracle. Kimia represents the implemented funding update as a bounded carry-spread kernel:

$$\mathfrak{F}_K(t) = \frac{\text{clamp}(\overline{M}_t - \overline{O}_t + \overline{O}_t/c, -O_t/c_{\max}, O_t/c_{\max})}{\overline{O}_t P} \cdot 10^9 \quad (1)$$

where  $\overline{M}_t$  and  $\overline{O}_t$  are the mark and oracle TWAPs,  $O_t$  is the current oracle price,  $c = 5000$  is the carry denominator,  $c_{\max} = 33$  is the clamp denominator, and  $P = 24$  is the number of funding periods per day. The formula is useful as a Kimia implementation specification: it names the exact integer-scaled quantity accumulated into the market’s long and short cumulative funding fields.

Settlement is lazy and incremental. Each user account stores its last-observed cumulative-rate snapshot; the next user instruction computes a signed funding delta, converts it across base and quote precision, and debits or credits collateral before continuing.

### 2.2 Delta-Neutral Hedging

A short perp position with size  $-q$  paired with a spot long of  $+q$  reduces first-order price exposure while preserving the funding leg. Kimia’s contribution is the on-chain instantiation: both legs live inside the same Solana execution surface, and the vault crosses CPI into `kimia-perp` to open, settle, and rebalance the hedge.

### 2.3 The Reserved-Margin Health Model

Kimia’s account-health convention reserves margin at placement time. When a user submits a resting limit order, the engine immediately debits the implied initial-margin requirement from the user’s free-collateral pool. The free balance available for the next trade is represented by the Kimia reserved-balance map:

$$C_{\text{free}}(u) = \text{collateral}(u) - \sum_{o \in \text{orders}(u)} m_{\text{init}}(o) \quad (2)$$

This is the “already spent” model: a position and an unfilled order compete for the same collateral, so a trader cannot over-leverage by placing many resting orders that would all clear simultaneously. Kimia uses the stricter placement-time reserve because Solana’s parallel execution and orderbook cross-referencing make eventual consistency under simultaneous fills harder to guarantee.

The liquidation health used by the cranker includes lazy funding inside the buffer:

$$H(u) = C_{\text{free}}(u) - \pi_{\text{pending}}(u) + \text{UPnL}(u) - m_{\text{maint}}(u), \quad H(u) \leq 0 \Rightarrow \text{liquidatable} \quad (3)$$

Including  $\pi_{\text{pending}}$  closes a corner case where a user could otherwise defer settlement of an underwater position by withholding the funding-settlement instruction. In the paper we treat Eq. 3 as Kimia’s liquidation predicate, not as a borrowed pricing expression.

## 2.4 The Split-Reward High-Water Guard

Vault shares accrue value as the underlying funding stream grows the vault’s NAV. The split engine tracks the vault share-price high-water mark and does not lower the baseline during drawdowns. A future reward vault can convert positive share-price movement into distributable YT rewards under Kimia’s guarded accumulator:

$$R_{t+1} = R_t + \max(0, P_{t+1} - P_t) \cdot \frac{\Sigma_{\text{share}}}{\Sigma_{YT}} \quad (4)$$

Negative-funding periods can drag the vault’s share price below a previous high-water mark. The guarded form prevents a later recovery from being misclassified as fresh distributable yield. In the deployed split-engine code, `update_rewards` advances the high-water share-price marker; a dedicated reward vault is required before YT rewards can be paid as assets.

## 2.5 Implementation-Side AMM Invariant Guard

Kimia’s AMM implementation is specified by a guarded reserve score rather than by an unguarded closed-form quote. Let  $\Phi(y, \tau)$  denote the integer fixed-point exponentiation routine used on chain. For reserves  $(x, y)$  and time factor  $\tau$ , the program scores pool solvency as

$$\mathcal{K}_K(x, y, \tau) = \lfloor x \cdot \Phi(y, \tau) \rfloor_{10^9} . \quad (5)$$

Because  $\Phi$  is implemented with bounded integer  $\ln / \exp$  approximations, the swap path never trusts a quote by itself. After computing the candidate output  $\Delta y$ , the engine recomputes the reserve score on the post-swap reserves and applies the rule

$$\text{while } \mathcal{K}_K(x', y', \tau) < \mathcal{K}_K(x, y, \tau) : \quad \Delta y \leftarrow \Delta y - 1 \quad (6)$$

i.e. it decrements the trader’s output by one base unit until the post-swap reserve score is non-decreasing. This is the Kimia-specific safety net: LP solvency is enforced by the actual integer transition that lands on chain, not by a symbolic curve drawn in real numbers.

## 2.6 The Pre-Pause Solvency Predicate

The vault’s automatic pause is governed by a two-condition predicate evaluated on every state-changing instruction:

$$\text{Pause}(t) = (\text{Insurance}(t) = 0) \wedge (t - t_{\text{neg-start}} \geq 86,400 \text{ s}) \quad (7)$$

where  $t_{\text{neg-start}}$  is the timestamp at which negative funding was first detected without recovery. The 24-hour duration is the on-chain cooling threshold: short enough to react to sustained stress, long enough to absorb intra-day spikes. The choice is implementation-specific to Kimia and should be treated as a risk parameter rather than a universal solvency rule.

## 3 Protocol Architecture

Kimia is decomposed into six Anchor [11] programs on Solana [10], each holding a single responsibility and communicating exclusively through cross-program invocation (CPI). No program imports another’s mutable state; all coupling is over typed instruction interfaces and SPL Token flows. Generated TypeScript clients are produced from the Anchor IDLs.

| Program       | Responsibility                                     | Program ID (devnet) |
|---------------|----------------------------------------------------|---------------------|
| kimia-perp    | Matching engine, funding, liquidation, spot pool   | 56jfo...vrK1n       |
| delta-vault   | USDC vault, paired spot+perp hedge, NAV accounting | HyCt6...TuJ2        |
| split-engine  | PT/YT minting, high-water reward accounting        | CKnwf...NF36q       |
| yield-amm     | PT $\leftrightarrow$ underlying yield-space AMM    | 7d48W...VCT9e       |
| intent-router | Multi-tx rate-target session state machine         | CU3A8...yyjR6       |
| kusd-mint     | Multi-reserve stablecoin and staking yield         | 94kLq...vkTZ2       |

Table 1: Program responsibility map. Each program is independently auditable.

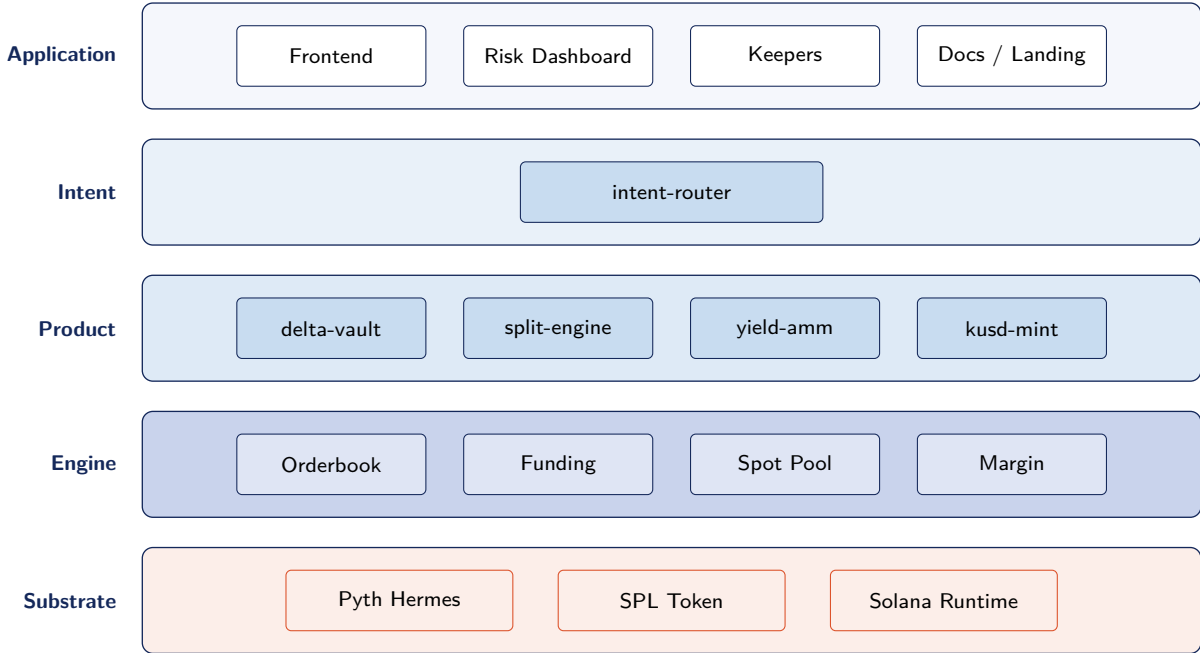


Figure 1: Five-layer architecture. Each layer depends only on layers strictly below it. The intent-router sits above the product layer because it composes products; the kimia-perp engine is the core, depending only on the substrate layer.

### 3.1 Layered View

### 3.2 CPI Coupling

Inter-program coupling is entirely explicit, expressed as CPI calls with feature-gated entrypoints: The graph is intentionally a DAG: no program calls a program that depends on it transitively. This eliminates re-entrancy risk by construction and means each program can be deployed, audited, and upgraded in isolation.

## 4 Perpetual Engine: `kimia-perp`

The matching engine is the bottom of the stack. It supports a single-collateral (USDC) margin model, a fully on-chain price-time priority orderbook (in the spirit of Phoenix [8]), lazy funding settlement [1], and an oracle-priced spot pool used by the vault hedge leg. Pricing is sourced from Pyth Hermes pull oracles [9].

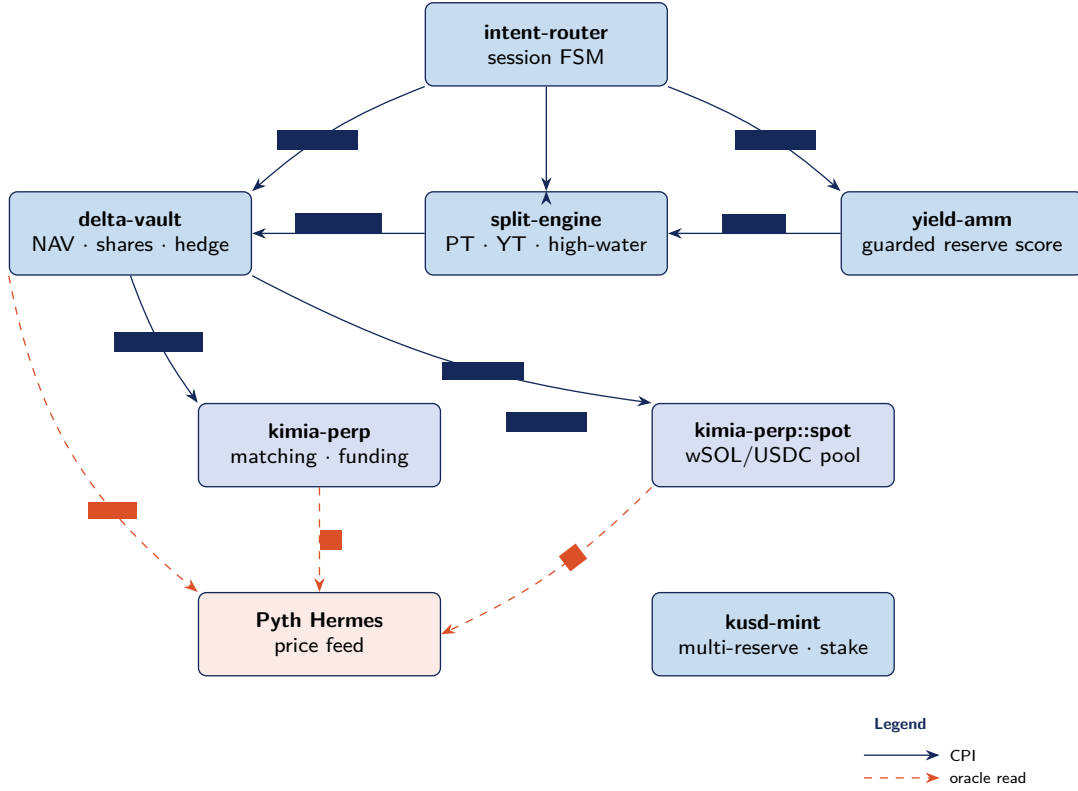


Figure 2: CPI graph. Solid arrows are program-to-program CPI; dashed arrows are oracle reads. The graph is a strict DAG: no program calls a program that depends on it transitively. `kUSD-mint` is a sibling settlement and staking product, not part of the yield-tokenization execution path.

#### 4.1 Position Accounting

Each user maintains a single `UserAccount` PDA per market with the fields summarized in Table 2.

| Field                                | Type           | Semantics                                   |
|--------------------------------------|----------------|---------------------------------------------|
| <code>collateral</code>              | $u_{64}$ (1e6) | USDC margin balance                         |
| <code>base_amount</code> ( $b$ )     | $i_{64}$ (1e9) | Signed position; positive = long            |
| <code>quote_entry</code> ( $q_e$ )   | $i_{64}$ (1e6) | Cumulative quote cost; sign opposite to $b$ |
| <code>last_cumulative_funding</code> | $i_{128}$      | Last settled cumulative funding rate        |
| <code>total_reserved_margin</code>   | $u_{64}$ (1e6) | Margin locked by resting orders             |

Table 2: User account fields.

Unrealized PnL is evaluated from Kimia’s signed-position tuple  $(b, q_e)$ : base amount  $b$  is signed, and `quote_entry` is a signed cumulative quote balance (negative for longs that paid USDC, positive for shorts that received USDC). The result is branch-free at read time because both sides share the same storage convention.

#### 4.2 Reserved-Margin Convention

Kimia uses the strict “already-spent” margin model defined in §2.3. The free balance available for new trades is the user’s collateral minus the running sum of initial-margin requirements across all resting orders. Computing only against a post-fill hypothetical can admit a race condition under Solana’s parallel execution where two simultaneously-validated orders each



pass individually yet collectively over-leverage the account. The reserved-margin convention removes the race by treating placement, not fill, as the moment collateral becomes unavailable.

### 4.3 Mark Price Clamp

The mark price is the orderbook last-trade price bounded to a  $\pm 10\%$  window around the oracle, with the oracle itself as the fallback when no trade has occurred. The window is implementation-specific: it is wide enough to admit organic basis (which is what feeds the funding-rate calculation) but narrow enough that a single thin-liquidity print cannot manipulate funding by an order of magnitude. The clamp is applied symmetrically and unconditionally; we do not allow the bound to widen during stress. This addresses the MEV-style attack surface documented for naive last-price marks [12].

### 4.4 Liquidation

Liquidations are permissionless. A liquidation fee of up to 5% of the closed notional is taken from positive account equity and split between the liquidator and the insurance fund. If the account is already underwater, the fee is capped by available equity and bad debt is handled through the insurance path. The closed position is taken at the oracle price (not the mark) to prevent predatory liquidators from steering the orderbook against an underwater account.

### 4.5 Orderbook

The orderbook is a single zero-copy account holding a fixed-capacity, sorted, two-sided book. Bids are stored in descending price order; asks in ascending. Each side admits a maximum of 32 resting orders; this is a deliberate ceiling so the entire book fits within Solana’s stack and a single fill iteration can scan both sides without exceeding the compute-unit budget.

| Field                        | Bytes | Purpose                                |
|------------------------------|-------|----------------------------------------|
| <code>owner</code>           | 32    | Authority of the resting order         |
| <code>order_id</code>        | 8     | Monotonic global ID for cancel routing |
| <code>price</code>           | 8     | Tick-aligned limit price               |
| <code>base_remaining</code>  | 8     | Unfilled size in $10^9$ units          |
| <code>reserved_margin</code> | 8     | Initial margin locked at placement     |
| <code>direction</code>       | 1     | 0 = bid, 1 = ask                       |
| <code>is_active</code>       | 1     | 1 = slot occupied                      |

Table 3: Orderbook entry layout. 67 bytes per slot under Pubkey alignment, 32 slots per side,  $\sim 4.4$  KB total.

A market order `place_order` with `side = Buy` scans the asks ascending until the requested fill is exhausted or the book runs out:

**Algorithm 1** Market-order match (taker-side, simplified)

---

```

1: filled  $\leftarrow$  0; quote  $\leftarrow$  0
2: for each ask  $a_i$  in ascending price order do
3:   if  $a_i.\text{price} > \text{limit}$  then
4:     break
5:   end if
6:   take  $\leftarrow \min(a_i.\text{base\_remaining}, \text{size} - \text{filled})$ 
7:   filled  $\leftarrow$  filled + take; quote  $\leftarrow$  quote + take  $\cdot a_i.\text{price}$ 
8:   settle PnL on  $a_i.\text{owner}$ , decrement  $a_i.\text{base\_remaining}$ 
9:   if  $a_i.\text{base\_remaining} = 0$  then
10:    remove slot  $i$ 
11:   end if
12:   if filled = size then
13:     break
14:   end if
15: end for
16: debit taker fee, credit maker rebate, update mark, write user position

```

---

The maker accounts of every fill must be passed in `remaining_accounts`, in book order. This is unusual but necessary on Solana: the program must hold a writable handle to each account it mutates, and Solana does not expose dynamic account loads. Frontends pre-walk the book, sort, and pass the maker UserAccounts that will be touched.

## 5 Delta Vault

The delta vault is the funding-yield container. It accepts USDC, simultaneously opens a paired spot wSOL position and an offsetting SOL-PERP short via CPI into `kimia-perp`, and mints share tokens to the depositor.

### 5.1 Deposit Mechanics

For a deposit of  $K$  USDC with the hedge flag enabled, the vault executes atomically:

1. Transfer  $K$  USDC from depositor to vault.
2. Split:  $K/2$  stays as USDC reserve;  $K/2$  swaps to wSOL on the spot pool at oracle price  $\pm$  spread.
3. Open SOL-PERP short of size  $q = (K/2)/S_t$  via `place_order`, posting the corresponding initial margin from the USDC reserve.
4. Mint vault shares  $\Delta\Sigma$  to the user where  $\Delta\Sigma = K \cdot \Sigma_t / \text{NAV}_t$  (or  $K$  if  $\Sigma_t = 0$ ).

After the deposit, the spot leg's price exposure is offset by the perp leg, leaving only funding flows in NAV. The 50/50 split between spot reserve and perp margin is deliberate: it gives the vault exactly enough wSOL to neutralize the perp short notional at the deposit price, and enough free USDC to cover the initial margin and a buffer for funding settlement.

### 5.2 NAV and Share Price

The vault tracks NAV explicitly and reconciles it on every state-changing instruction. Reconciliation is incremental: each instruction reads the current `kimia-perp realized_pnl` for the

vault's `UserAccount`, computes the delta against a stored snapshot, applies the insurance skim, and writes back to NAV. Withdrawals are netted directly. The result is a single source of truth for share price NAV divided by total share supply, in fixed-point that does not require an off-chain oracle and is provably consistent with the funding payments that have actually settled on the perp engine.

This reconciliation is robust against partial CPI failures because the vault re-reads `kimia-perp` state *after* the CPI returns. If a CPI partially failed and was rolled back, the snapshotted PnL is unchanged and NAV is untouched. If it succeeded, NAV reflects the new realized state. The vault does not maintain a parallel ledger of expected funding; it always defers to what the perp engine has actually accounted, which removes the entire class of bugs that arise from divergence between expected and realized vault state.

### 5.3 Rebalancing and Pause

The vault is permissionlessly rebalancable: any caller may invoke `rebalance`, which compares the normalized notional gap between the spot and perp legs against the per-vault threshold `rebalance_threshold_bps`. If the gap exceeds the threshold, the vault closes half of it via spot swap (when `spot > perp`) or perp buyback (when `perp > spot`). Otherwise the call is a no-op that emits a `RebalanceChecked` event. The half-step is deliberate: closing the entire gap in a single transaction would amplify slippage and, under adverse oracle moves between rebalances, potentially open the gap on the opposite side.

The vault auto-pauses under the predicate of Eq. 7; in `Paused`, deposits are blocked and only the admin unwind path is reachable.

## 6 Split Engine

The split engine wraps a vault share into a Principal Token (PT) and a Yield Token (YT), following the tokenization pattern pioneered by Element [5], refined by Pendle [2], and deployed at scale in Notional [4]. One PT plus one YT can always be re-combined into one share, but each can be transferred and traded independently.

### 6.1 Mint and Redeem

A market is parameterized by a maturity  $T$ . Deposit of  $\Delta$  vault shares at time  $t < T$  mints  $\Delta$  PT and  $\Delta$  YT atomically. The shares are escrowed in a market-owned token account. After maturity ( $t \geq T$ ), holders may redeem one PT for one vault share via `redeem_pt`; YT no longer receives new accrual after maturity.

Before maturity, the market records user YT balances and reward debt, giving the program the accounting surface needed for YT reward distribution. The current implementation is intentionally conservative: `update_rewards` advances the vault share-price high-water mark and avoids treating recovery from a drawdown as new yield, while `claim_yield` does not transfer principal escrow to YT holders. A production reward vault can attach to the same accounting kernel and distribute only backed rewards under Eq. 4.

### 6.2 Early Exit

A holder who has both PT and YT in equal quantities may invoke `early_exit`, which burns both and returns the underlying share minus a 30 bps fee. This is the canonical reverse of the

deposit. It enables liquidity providers and arbitrageurs to neutralize a position without trading on the AMM.

## 7 Yield-Space AMM

The AMM trades PT against the underlying vault share. Its purpose is to provide a continuous price for  $PT$  that decays into par at maturity, so PT holders can crystallize a discount rate at any time before  $T$ . Kimia’s on-chain specification is the guarded reserve score of Eq. 5: quotes are accepted only if the post-swap integer transition preserves or improves that score. The resulting market borrows the economic intuition of time-decay yield pricing [3, 4, 2], but the paper’s operative formula is the Kimia guard applied to real on-chain reserves.

### 7.1 Pool Parameters

The pool is parameterized by the same maturity  $T$  as the underlying split-engine market and a creation time  $t_0$ . Reserves are  $(x, y)$  for underlying and PT. The Kimia maturity clock

$$\tau_K(t) = \frac{\max(0, T - t)}{T - t_0}$$

shrinks linearly to zero at maturity and is passed into Eq. 5. A constant 30 bps swap fee is deducted from the input before quote validation; the fee remains in the pool and is captured by LPs.

### 7.2 Time Decay and Convergence

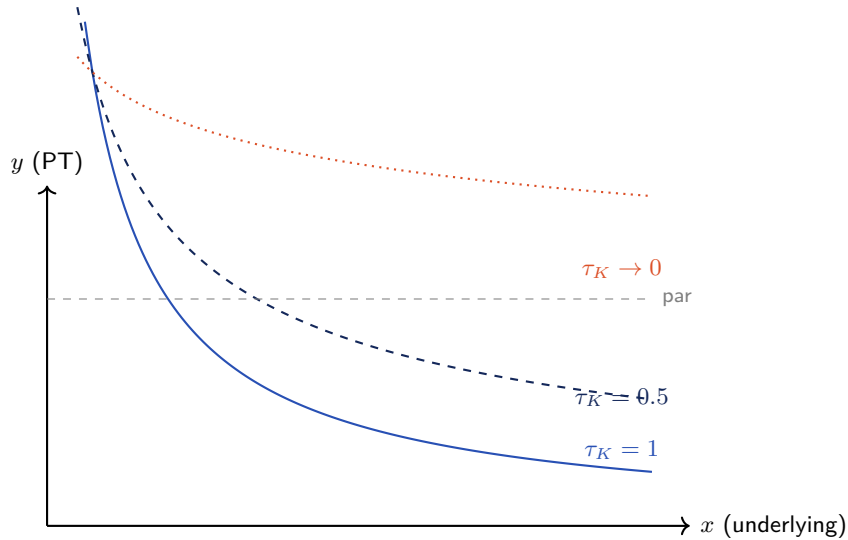


Figure 3: Kimia AMM reserve-score contours for three values of the maturity clock. The diagram is illustrative: the on-chain rule is the integer score guard in Eq. 5 and Eq. 6, not an unconstrained real-number curve. As  $\tau_K \rightarrow 0$ , PT trades approach par.

The clock  $\tau_K$  shrinks linearly to zero as  $t \rightarrow T$ . The AMM disables swaps once the clock falls below the near-maturity floor, leaving redemption as the primary path after maturity. This avoids asking an integer AMM to price vanishing-tenor trades where one base unit of rounding can dominate the quoted rate.

### 7.3 Implementation Notes

The reserve score requires fractional exponentiation. `kimia-math` implements  $\Phi(b, e)$  via:

- range-reduced natural log on  $[0.5, 2.0)$  with the  $z = (x-1)/(x+1)$  series, accurate to  $< 0.05\%$ ;
- exponential via  $e^x = e^r \cdot 2^n$  with  $|r| < \ln 2/2$ , Taylor-expanded over twelve terms.

The quote is verified post-execution by recomputing  $\mathcal{K}_K$  on the new reserves; if rounding produced a quote that decreases the score, the engine decrements the output by one unit and re-checks until  $\mathcal{K}_{K,\text{after}} \geq \mathcal{K}_{K,\text{before}}$ . This guarantees the guarded reserve score is non-decreasing across swaps.

## 8 Intent Router

The intent router is a session state machine that wraps the four-step yield-tokenization flow (deposit  $\rightarrow$  split  $\rightarrow$  swap  $\rightarrow$  verify) into a single user-facing primitive with a target-rate guard.

### 8.1 Session State Machine

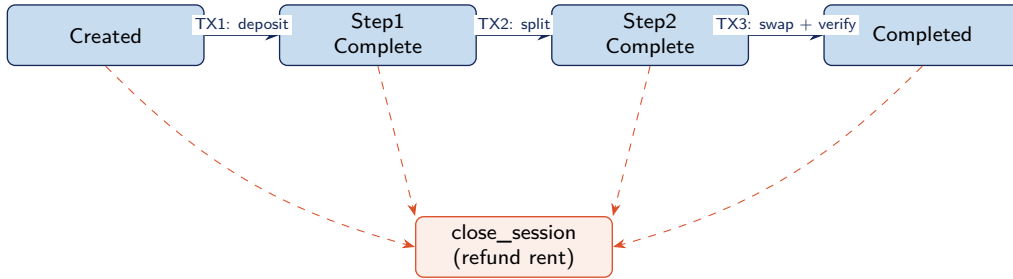


Figure 4: Intent session state machine. Solid arrows are forward transitions on each TX; dashed arrows close the session at any point and refund the PDA rent.

The session PDA is seeded with  $(\text{owner}, \text{session\_id})$  and stores the target rate  $R^*$ , duration  $\Delta T$ , original deposit  $K$ , intermediate balances after each step, the chosen split-engine market, and the AMM pool. Each step's `record_step_i` instruction validates the previous state and writes the result of the corresponding TX. The verification step recomputes the annualized discount rate from the PT sold and the underlying received, then reverts if it falls below  $R^*$ .

### 8.2 Why a State Machine?

The yield-tokenization flow involves three CPI hops touching different programs. Compressing it into one transaction would exceed Solana's compute budget for non-trivial fill paths. The router instead persists progress across transactions, so each TX is small and self-contained. The tradeoff is that the intermediate state is publicly visible; this is acceptable because the output (a final balance check) is what defines correctness.

## 9 kUSD Stablecoin Layer

`kusd-mint` is a multi-collateral stablecoin and settlement layer. It supports registered backing stables, cross-stable swap intents, and a USDC staking-reward channel funded by an authorized

yield authority. This makes kUSD a distribution and settlement surface for protocol yield without assuming that every fee path is automatically routed into stakers.

## 9.1 Mint and Redeem

kUSD is collateralized at 1:1 by registered backing stables. The first reserve at genesis is USDC; additional reserves (e.g. `kamino.usd`, `drift.usd`) are added by admin with a per-reserve weight cap  $w_i$  in basis points. The total deposited collateral across all reserves must always meet or exceed total kUSD outstanding, and no single reserve may back more than  $w_i/10^4$  of the global cap. Mint and redeem are 1:1, with the user choosing which backing stable to deposit or receive.

## 9.2 Cross-Stable Swap Intents

A user wishing to swap one backing stable for another (say `kamino.usd`  $\rightarrow$  `drift.usd`) creates a `SwapIntent`. The protocol issues kUSD immediately and locks the input stable. T+1, after off-chain rebalancing has been settled, an admin crank marks the intent settled, and the user claims their target stable, burning the kUSD. This decouples user-facing instant settlement from the slow reserve rebalancing required to keep weight caps in line.

## 9.3 Staking Yield

Users stake kUSD in a global `StakePool` to earn USDC rewards funded by the yield authority (typically the protocol's own delta-vault funding stream). The staking pool uses Kimia's 18-decimal reward scalar so USDC rewards can be distributed across 6-decimal deposits without losing precision. The reserve-yield path (`deploy_reserves`, `pull_vault_yield`) tracks idle USDC lent to external strategies and surfaces returns through the same staking-accounting surface.

# 10 Risk Engine and Insurance

The risk engine is an off-chain TypeScript service that decodes `kimia-perp`, `delta-vault`, and `kusd-mint` accounts directly from RPC bytes. It does not trust the frontend or any indexer.

## 10.1 Health Snapshot

For each open position, the engine decodes account state and surfaces a normalized health factor (equity over maintenance margin); the CLI monitor logs warnings at  $HF < 1.5$  and the dashboard tabulates the population by quantile. The on-chain liquidation path remains authoritative because it includes pending lazy funding inside Eq. 3. The off-chain implementation is independent of the frontend and uses only RPC byte-decoding from `kimia-perp`, `delta-vault`, and `kusd-mint` accounts.

## 10.2 Insurance and Crank Incentives

Three permissionless cranks keep the system in lockstep: `update_funding_rate` (once per hour), `liquidate` (any time an account violates Eq. 3), and `update_rewards` (synchronizes the split-engine high-water share-price marker). Liquidation has a direct economic payout when positive account equity is available; funding updates, reward updates, and vault rebalances are keeper

services whose incentive is indirect or bundled with other transactions. The insurance fund absorbs residual bad debt where possible; if the vault insurance reserve is depleted under sustained negative funding, the vault auto-pauses under Eq. 7.

## 11 Implementation Notes

This section collects practical decisions whose rationale is not visible from the math alone but is forced by Solana's execution model.

### 11.1 Account Loading and `remaining_accounts`

Solana programs receive the full set of accounts they will read or write inside the instruction at dispatch time. Dynamic loads are not supported. For `kimia-perp::place_order`, this means every `UserAccount` that will be touched by the fill must be enumerated up front. Frontends therefore pre-walk the orderbook off-chain, compute the set of maker accounts that the order will sweep against, and pass them as `remaining_accounts` in book order. The program validates the pubkeys against the maker entries it actually consumes; any missing or out-of-order account aborts the transaction.

The same pattern shows up in `delta-vault::deposit` when `open_perp = true`: the vault is the taker, and the maker `UserAccounts` that will be filled against must be threaded in. This pushes some matching responsibility off-chain frontends and keepers cooperate to surface a workable maker set in exchange for a deterministic on-chain control flow.

### 11.2 Zero-Copy Orderbook

The orderbook account is approximately 4.4 KB, larger than Solana's program stack will reasonably accommodate as a regular Anchor account. We use Anchor's zero-copy mechanism: the account is treated as a borrowed memory region rather than a deserialized struct, and individual field accesses are unaligned reads against the raw account data. This avoids stack allocation entirely. The trade-off is that mutations must be careful with field types for example, the `is_active` field is a `u8` rather than a `bool`, because zero-copy does not natively support `bool`.

### 11.3 PDA Authority and CPI Signing

Every cross-program call from `delta-vault` into `kimia-perp` signs as the vault's PDA. The vault PDA is the authority on the vault's USDC token account, the wSOL token account, and the perp-perp `UserAccount` that holds the hedge position. The perp engine accepts the vault's CPI through a `delegate` field on the `UserAccount` that allows a non-authority signer to trade but not to withdraw. The vault is the authority and the keeper is the optional delegate; this keeps custody on-chain (the vault PDA) while permitting permissionless cranking.

### 11.4 Lazy Funding and Settlement Order

Funding accrual is split into two pieces: a global update (`update_funding_rate`) that advances the cumulative-rate scalars, and a per-user settle (`settle_funding`) that applies the user-specific delta to collateral. Both are permissionless. The lazy split keeps the global update cheap (one instruction touches one Market account) and lets the per-user cost be paid only when a user is actually transacting. The on-chain liquidation health predicate (Eq. 3) reads pending funding

directly, so a malicious actor cannot avoid liquidation by withholding their own settlement instruction.

## 11.5 Rounding Discipline

All user-facing arithmetic rounds against the trader. Multiplications and divisions in `perps-math::safe_math` expose an `_up` variant for cases where rounding up is conservative (margin requirements, fees) and a `_down` variant for cases where rounding down is conservative (PnL credits, position sizes). The convention is enforced at the math-crate level rather than at each call site, so a single audit of `safe_math` is sufficient to verify direction-of-rounding correctness across all programs that depend on it.

## 11.6 Permissionless Cranking

Three categories of state advancement are permissionlessly callable:

- **Funding update.** Anyone may invoke `update_funding_rate` once per period; the engine reads the latest oracle, rebuilds the TWAPs, and accrues the cumulative scalars. The caller pays the transaction fee but receives no direct payout. Their incentive is indirect: every long with a positive position has a financial reason to settle their funding before opening more risk, and any party already paying compute on a transaction has near-zero marginal cost to crank funding alongside their primary instruction.
- **Liquidation.** Anyone may invoke `liquidate` on an account where Eq. 3 holds. The liquidator earns 50% of the 5% liquidation fee on the closed notional. The other 50% accrues to the insurance fund, providing a non-trivial budget for absorbing tail-loss events that close past the maintenance threshold.
- **Reward and rebalance cranks.** `update_rewards` on the split engine and `rebalance` on the delta vault are also permissionless. Neither pays the caller directly; both are gas-paid services that unsophisticated users can rely on sophisticated actors (keepers, market makers, arbitrageurs) to provide. The protocol’s design assumes at least one such actor exists per market.

The combined effect is that the protocol’s state can advance without any privileged authority. The admin authorities that exist (vault initialization, market parameter updates, kUSD reserve registration) are all configuration-time roles, not run-time gatekeepers.

## 12 Numerical Precisions

All on-chain math is integer. Floating point is forbidden in Solana program code. Kimia consolidates its scaling conventions in two crates: `perps-math` for matching-engine primitives and `kimia-math` for the products. The conversion table is in Table 4.

Conversions cross precisions explicitly. For example, the funding-settlement conversion divides by  $\text{FUNDING\_PRECISION} \cdot 10^3$  to align  $b$  (1e9) with the quote-side payment (1e6). All multiplications use  $u_{128}$  intermediates to prevent overflow; rounding is always against the trader at user-facing boundaries.



| Constant          | Value     | Domain                          |
|-------------------|-----------|---------------------------------|
| PRICE_PRECISION   | $10^6$    | USD prices (Pyth-aligned)       |
| QUOTE_PRECISION   | $10^6$    | USDC native units               |
| BASE_PRECISION    | $10^9$    | SOL lamports, vault shares      |
| MARGIN_PRECISION  | $10^4$    | Margin ratios in BPS-like units |
| FUNDING_PRECISION | $10^9$    | Funding rates                   |
| KIMIA_MATH_ONE    | $10^9$    | AMM fixed-point unit            |
| REWARD_PRECISION  | $10^{18}$ | kUSD staking reward-per-share   |

Table 4: Fixed-point conventions across crates.

## 12.1 Fixed-Point Math Library

The yield AMM requires fractional exponentiation, which is not natively available in integer arithmetic. `kimia-math::exp_ln` implements the integer routine  $\Phi(b, e)$  with range-reduced logarithm and exponential approximations. The important protocol-level formula is not the approximation method itself; it is the Kimia reserve-score guard in Eq. 5 and Eq. 6, which converts bounded numerical precision into a strict on-chain transition rule by decrementing the trader’s output until the score is non-decreasing.

## 13 End-to-End Yield-Tokenization Flow

We close the technical specification with a worked example of the canonical user flow.

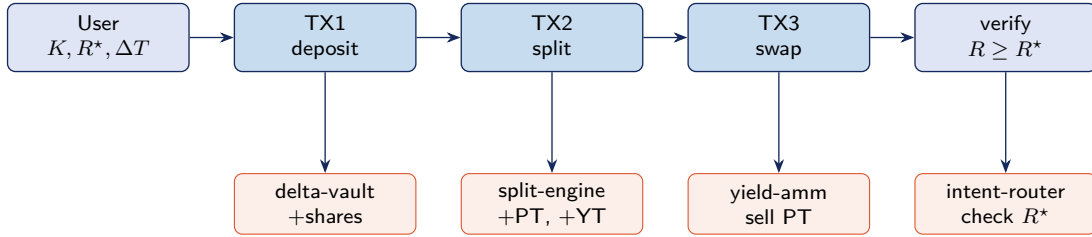


Figure 5: Yield-tokenization flow. The user signs three transactions; the router verifies the AMM-implied rate against  $R^*$  atomically inside TX3.

**Schematic example.** A user deposits  $K$  USDC, requests duration  $\Delta T$ , and sets target  $R^*$ . The user receives  $K$  vault shares (assuming unit share price at deposit), splits them into  $K$  PT and  $K$  YT, then sells some PT on the AMM for  $u$  underlying. The router verifies the claimed PT sold against the user’s post-swap PT balance and computes the achieved annualized discount rate from the PT face value and underlying received. If that rate is below  $R^*$ , the verification step reverts. The actual numbers depend on prevailing AMM reserves,  $\tau$ , and the swap fee, all of which are read at the time of TX3 and are not separately bound by the protocol.

## 14 Discussion

### 14.1 Why Solana

The protocol is gas-sensitive in three places: the orderbook’s per-fill cost, the AMM’s fractional exponentiation, and the multi-CPI intent flow. Solana’s parallel execution and sub-cent transaction fees make all three viable. The orderbook uses zero-copy account layouts to fit a 32-deep

book per side in a single 4.4 KB account, and the AMM’s  $\exp/\ln$  implementation completes a swap inside the standard 200K compute-unit budget.

## 14.2 Composition Surface

Each Kimia program is independently consumable. A protocol that wants only the funding-yield primitive can integrate `delta-vault` alone. A fixed-income protocol can integrate `split-engine` on top of any other yield-bearing share token (not just Kimia’s vault). The yield-space AMM is general; it works with any (PT, underlying) pair under a known maturity. The strict CPI DAG (Fig. 2) means external integrators inherit a small audit surface per use case.

## 14.3 Security Properties

We collect the protocol’s structural invariants in one place:

| Invariant                                                                                                    | Enforcement                                                                                               |
|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| $\mathcal{K}_{K,\text{after}} \geq \mathcal{K}_{K,\text{before}}$ on every AMM swap                          | Post-execution recomputation; rounded outputs decremented until satisfied.                                |
| $\sum_i C_u(i) \geq \sum_i m_{\text{init},u}(i)$                                                             | Reserved-margin debit at order placement; cancel/fill credits back.                                       |
| $\text{NAV}_t = \text{deposits} + \text{settled\_funding} - \text{insurance\_drawdown} - \text{withdrawals}$ | <code>admin_resync_nav</code> reconciles after CPI; checks against <code>kimia-perp.realized_pnl</code> . |
| $\text{PT} + \text{YT} = \text{share}$ at all times                                                          | Split engine mints atomically; early exit burns matched pairs.                                            |
| $\sum_i \text{deposited}_i \geq \text{total\_kUSD}$                                                          | Reserve weight caps and per-mint accounting.                                                              |
| oracle staleness < 60 s, confidence < 250 bps                                                                | Pyth read aborted otherwise.                                                                              |
| Rebalance gap reduction                                                                                      | Permissionless crank closes half the notional gap on any call where $ \delta_t  > \delta^*$ .             |

Table 5: Protocol invariants and where they are enforced.

## 14.4 Limitations

Three boundaries deserve explicit acknowledgement.

- *Funding regime risk.* The vault’s NAV grows in expectation under positive average funding. In a sustained negative-funding regime (a structural inversion), the insurance fund absorbs losses and the vault auto-pauses after 24 hours. This is a containment mechanism, not a hedge against negative funding itself; positions opened during such a regime will lose money.
- *AMM liquidity at maturity.* As  $\tau \rightarrow 0$ , the curve flattens and a small reserve imbalance can produce large slippage on dollar-denominated trades. The protocol mitigates this by disabling swaps when  $\tau$  falls below a configured floor (the `swap.rs` handler aborts with `SwapDisabledNearMaturity` when  $\tau < 10^{-3}$  in the current implementation); past that point, only redemption via `redeem_pt` after maturity remains.
- *Single-market scope.* V1 ships SOL-PERP only. The architecture is multi-market by design (every `Market` account is keyed by a distinct `market_index`), but additional markets require admin-gated bootstrapping and per-market oracle/risk parameters.

## 14.5 Position in the Landscape

Kimia’s component primitives sit near existing work: Pendle and Element [2, 5] for PT/YT tokenization, Notional and YieldSpace [4, 3] for time-decay yield pricing, Drift [1] for lazy-funding perpetuals, and Ethena [6] for the synthetic-dollar use case on perp basis. The contribution here is the *composition*: pairing on-chain delta-neutral funding capture with PT/YT tokenization, a guarded yield AMM, a rate-verifier intent router, and a stablecoin settlement layer, all coupled through CPI on Solana. The formulas in this paper are written as Kimia state-transition specifications and guard predicates, so the technical focus remains on what the implementation enforces.

## 15 Conclusion

Kimia turns the funding-rate yield of an on-chain perpetual into a delta-neutral, tokenizable, term-structured primitive. The construction relies on six independently auditable Anchor programs whose CPI graph is a strict DAG: a perpetuals matching engine produces the funding flow, a delta-neutral vault targets that flow while reducing price exposure, a split engine separates principal from yield, a yield-space AMM prices PT against time-to-maturity, an intent router verifies the AMM-implied discount rate against the user’s target, and a multi-collateral stablecoin layer supports settlement and reward distribution. We deliberately do not present the protocol as a guaranteed-yield product: the realized USDC value at maturity is determined by the actual NAV of the underlying delta-neutral vault, which is a function of realized funding flows over the tenor.

We have given the implementation-side specification of each layer, focusing on the mechanisms that are specific to Kimia’s execution model: the Kimia funding kernel (Eq. 1), the reserved-margin liquidation predicate that includes pending lazy funding (Eq. 3), the split-engine high-water guard for future reward accounting (Eq. 4), the guarded AMM reserve score (Eq. 5), the post-swap decrement rule (Eq. 6), and the time-windowed automatic vault pause (Eq. 7). The composition is what matters: a stack with each layer operating under its own narrow trust assumption and contributing one financial primitive.

## References

- [1] W. Liu and C. Leow, “Drift Protocol v2: A Decentralized Perpetual Swap Exchange on Solana,” Drift Labs Whitepaper, 2022. <https://docs.drift.trade>
- [2] TN. Lee, V. Nguyen, and L. Vuong, “Pendle V2: A Permissionless Yield-Trading Protocol,” Pendle Labs Whitepaper, 2023.
- [3] A. Niemerg, D. Robinson, and L. Livnev, “YieldSpace: An Automated Liquidity Provider for Fixed Yield Tokens,” Yield Protocol, 2020. <https://yield.is/YieldSpace.pdf>
- [4] T. Woodward and J. Wu, “Notional Finance v2: Decentralized Fixed-Rate Lending on Ethereum,” Notional Labs, 2021.
- [5] W. Aboagye, P. Bukov et al., “Element Protocol: A Marketplace for Fixed and Variable Yield,” Element Finance, 2021.
- [6] G. Young, “Ethena: The Internet Bond,” Ethena Labs Whitepaper, 2024.
- [7] H. Adams, N. Zinsmeister, M. Salem, R. Keefer, and D. Robinson, “Uniswap v3 Core,” Uniswap Labs Whitepaper, 2021.

- [8] Ellipsis Labs, “Phoenix: A Performant On-Chain Limit Order Book on Solana,” Phoenix v1 Documentation, 2023.
- [9] Pyth Network, “Pyth: A First-Party Financial Oracle,” Pyth Network Whitepaper, 2022.
- [10] A. Yakovenko, “Solana: A New Architecture for a High-Performance Blockchain,” Solana Foundation, 2017.
- [11] A. Ferrante et al., “Anchor: A Framework for Solana’s Sealevel Runtime,” Coral, 2021. <https://www.anchor-lang.com>
- [12] P. Daian, S. Goldfeder, T. Kell, et al., “Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges,” *IEEE Symposium on Security and Privacy*, 2020.
- [13] G. Angeris and T. Chitra, “Improved Price Oracles: Constant Function Market Makers,” *ACM Conference on Advances in Financial Technologies (AFT)*, 2020.
- [14] J. C. Hull, *Options, Futures, and Other Derivatives*, 11th edition, Pearson, 2021.